# Section 20 - The Standard Template Library (STL)

## Information

- Title: `The Standard Template Library (STL)`
- Authors: `Etienne KOA`

## Section Overview

- What is the STL
- Generic programming/Meta-programming
  - Preprocessor macros
  - Function templates
  - Class templates
- STL COntainers
- STL Iterators
- STL Algorithms
- Array
- Vector
- Deque
- List and Forward List
- Set and Multi Set
- Map and Multi Map
- Stack and Queue
- Priority Queue
- Algorithms

---

- Qu'est-ce que la STL
- Programmation générique/Méta-programmation
  - Macros du préprocesseur
  - Modèles de fonctions
  - Modèles de cours
- Conteneurs STL
- Itérateurs STL

- Algorithmes STL
- Tableau -Vecteur
- Dequé
- Liste et liste de transfert
- Ensemble et ensemble multiple
- Carte et Multi-Carte
- Pile et file d'attente
- File d'attente de priorité
- Algorithmes

## Qu'est-ce que la `STL` ?

- Une bibliothèque de classes et de fonctions génériques puissantes, réutilisables, adaptables
- Implémenté à l'aide de modèles `C++`
- Implémente des structures de données et des algorithmes communs
- Immense bibliothèque de classe !!
- Alexandre Stepanov (1994)

### Pourquoi utiliser le `STL` ?

- Assortiment de contenants couramment utilisés
- Complexité connue en termes de temps et de taille
- Essayé et testé - Réutilisabilité !!!
- Cohérent, rapide et sécurisé
- Extensible

### Elements of the STL

- Conteneurs
  - Collections d'objets ou de types primitifs (array, vector, deque, stack, set, map, etc.)
- Algorithmes
  - Fonctions de traitement des séquences d'éléments issus de conteneurs (find, max, count, accumulate, sort, etc.)
- Itérateurs
  - Générer des séquences d'éléments à partir de conteneurs (forward, reverse, by value, by reference, constant, etc.)

### Un exemple simple

### Un exemple simple : trier un vecteur

### Un exemple simple : inverser un vecteur

### Un exemple simple : accumuler

```
#include <vector>
#include <algorithm>

std::vector<int> v {1,5,3};
```

Figure 1: STL

```
std::sort(v.begin(), v.end());

for (auto elem: v)
   std::cout << elem << std::endl;

1
3
5
```

Figure 2: STL

```
std::reverse(v.begin(), v.end());

for (auto elem: v)
   std::cout << elem << std::endl;

5
3
1
```

Figure 3: STL

```
int sum{};

sum = std::accumulate(v.begin(), v.end(), 0);
std::cout << sum << std::endl;

9   // 1+3+5
```

Figure 4: STL

### Type de conteneurs

- Conteneurs de séquence
  - array, vector, list, forward_list, deque
- Conteneurs associatifs
  - set, multi, set, map, multi map
- Adaptateurs de conteneur
  - stack, queue, priority queue

### Type d'itérateurs

- Itérateurs d'entrée - du conteneur au programme
- Itérateurs de sortie - du programme au conteneur
- Itérateurs avant - naviguez un élément à la fois dans une direction
- Itérateurs bidirectionnels - naviguez un élément à la fois dans les deux sens
- Itérateurs à accès aléatoire - accédez directement à un élément de conteneur

### Type d'algorithmes

- Environ 60 algorithmes dans la STL
- Non modifiable
- Modification

## Programmation générique avec des macros

- Programmation générique

  `Écrire du code qui fonctionne avec une variété de types d'arguments, à condition que c`

- Macros ***attention***
- Modèles de fonctions
- Modèles de cours

### Macros (`#define`)

- Directives du préprocesseur `C++`
- Aucune information sur le type
- Distribution simple

```
#define MAX_SIZE 100

#define PI 3.14159
```

### Fonction `max`

- Supposons que nous ayons besoin d'une fonction pour déterminer le « max » de 2 « entiers ».
- Supposons maintenant que nous devions déterminer le « maximum » de 2 « doubles » et de 2 « caractères ».

```
//#define MAX_SIZE 100      // removed

//#define PI 3.14159        // removed

if (num > 100)
    std::cout << "Too big");

double area = 3.14159 * r * r;
```

Figure 5: STL

```
int max(int a, int b) {
   return (a > b) ? a : b;
}

int x = 100;
int y = 200;
std::cout << max(x, y);      // displays 200
```

Figure 6: STL

```
int max(int a, int b) {
   return (a > b) ? a : b;
}

double max(double a, double b) {
   return (a > b) ? a : b;
}

char max(char a, char b) {
   return (a > b) ? a : b;
}
```

Figure 7: STL

**Macors avec arguments (`#define`)** + Nous pouvons plutôt écrire une macro générique avec des arguments.

```
#define MAX(a, b) ((a > b) ? a : b)

std::cout << MAX(10,20)    << std::endl;   // 20
std::cout << MAX(2.4, 3.5) << std::endl;   // 3.5
std::cout << MAX('A', 'C') << std::endl;   // C
```

Figure 8: STL

- Il faut faire attention aux macros

```
#define SQUARE(a) a*a

result = SQUARE(5);          // Expect 25
result = 5*5;                // Get 25

result = 100/SQUARE(5);      // Expect 4
result = 100/5*5             // Get 100!
```

```
#define SQUARE(a) ((a)*(a)) // note the parenthesis

result = SQUARE(5);          // Expect 25
result = ((5)*(5));          // Still Get 25

result = 100/SQUARE(5);      // Expect 4
result = 100/((5)*(5));      // Now we get 4!!
```

## Programmation générique avec des modèles de fonctions

- Qu'est-ce qu'un modèle `C++` ?
  - Plan
  - Modèles de fonctions et de classes
  - Autoriser le « pluggin-in » pour tout type de données
  - Le compilateur génère la « fonction/classe » appropriée à partir du plan
  - Programmation `Générique`/méta-programmation

**Revoyons la fonction `max` de la dernière conférence**

**Fonction `max`**

```
int max(int a, int b) {
   return (a > b) ? a : b;
}

int x = 100;
int y = 200;
std::cout << max(x, y);      // displays 200
```

Figure 9: STL

- Supposons que nous ayons besoin d'une fonction pour déterminer le « max » de 2 « entiers ».

```
int max(int a, int b) {
   return (a > b) ? a : b;
}

double max(double a, double b) {
   return (a > b) ? a : b;
}

char max(char a, char b) {
   return (a > b) ? a : b;
}
```

Figure 10: STL

**Fonction `max` comme fonction de modèle**

- Nous pouvons remplacer le type que nous voulons généraliser par un nom, disons `T`.
- Mais maintenant, cela ne sera pas compilé.

```
T max(T a, T b) {
   return (a > b) ? a : b;
}
```

Figure 11: STL

- Nous devons indiquer au compilateur qu'il s'agit d'une fonction modèle.
- Nous devons également lui dire que `T` est le modèle.
- Nous pouvons également utiliser `class` au lieu de `typename`

```
template <typename T>
T max(T a, T b) {
   return (a > b) ? a : b;
}
```

Figure 12: STL

```
template <class T>
T max(T a, T b) {
   return (a > b) ? a : b;
}
```

Figure 13: STL

- Le compilateur peut désormais générer la fonction appropriée à partir du modèle.
- Notez que cela se produit au moment de la compilation !

```
int a {10};
int b {20};

std::cout << max<int>(a, b);
```

Figure 14: STL

- Plusieurs fois, le compilateur peut déduire le type et le paramètre de modèle n'est pas nécessaire.
- En fonction du type de `a` et `b`, le compilateur le déterminera.
- Et nous pouvons utiliser « presque » n'importe quel type dont nous avons besoin.
- Notez que le type DOIT prendre en charge l'opérateur `>` soit de manière native, soit en tant qu'opérateur surchargé `(opérateur >)`
- Ce qui suit ne sera pas compilé à moins que `Player` surcharge `operator>`

**plusieurs types comme paramètres de modèle**

- Nous pouvons avoir plusieurs paramètres de modèle

```
std::cout << max<double>(c, d);

std::cout << max(c, d);
```

Figure 15: STL

```
char a {'A'};
char b {'Z'};

std::cout << max(a, b) << std::endl;
```

Figure 16: STL

```
template <typename T>
T max(T a, T b) {
   return (a > b) ? a : b;
}
```

Figure 17: STL

```
Player p1{"Hero", 100, 20};
Player p2{"Enemy", 99, 3};

std::cout << max<Player>(p1, p2);
```

Figure 18: STL

- Et leurs types peuvent être différents

```
template <typename T1, typename T2>
void func(T1 a, T2 b) {
   std::cout << a << " " << b;
}
```

Figure 19: STL

- Lorsque nous utilisons la fonction, nous fournissons les paramètres des modèles
- Souvent le compilateur peut les déduire

```
func<int,double>(10, 20.2);

func('A', 12.4);
```

Figure 20: STL

## Programmation générique avec modèle de classe

### Qu'est-ce qu'un Tempale `C++ Class` ?

- Similaire au modèle de fonction, mais au niveau de la classe
- Permet de « brancher » n'importe quel type de données
- Le compilateur génère la classe appropriée à partir du plan.

- Disons que nous voulons qu'une classe contienne des éléments où l'élément a un nom et un entier.

```
class Item {
private:
    std::string name;
    int value;
public:
    Item(std::string name, int value)
       : name{name}, value{value}
    {}
    std::string get_name() const {return name; }
    int get_value() const { return value; }
};
```

Figure 21: STL

- Mais nous aimerions que notre classe `item` puisse conserver tout type de données en plus de la chaîne.
- Nous ne pouvons pas surcharger les noms de classe
- Nous ne voulons pas utiliser le polymorphisme dynamique.

```
class Item {
private:
    std::string name;
    T value;
public:
    Item(std::string name, T value)
       : name{name}, value{value}
    {}
    std::string get_name() const {return name; }
    T get_value() const { return value; }
};
```

```
template <typename T>
  class Item {
  private:
      std::string name;
      T value;
  public:
      Item(std::string name, T value)
         : name{name}, value{value}
      {}
      std::string get_name() const {return name; }
      T get_value() const { return value; }
  };
```

```
Item<int> item1 {"Larry", 1};

Item<double> item2 {"House", 1000.0};

Item<std::string> item3 {"Frank", "Boss"};

std::vector<Item<int>> vec;
```

### Plusieurs types comme paramètres de modèle

- Nous pouvons avoir plusieurs paramètres de modèle
- Et leurs types peuvent être différents

```
template <typename T1, typename T2>
struct My_Pair {
    T1 first;
    T2 second;
};


My_Pair <std::string, int> p1 {"Frank", 100};

My_Pair <int, double> p2 {124, 13.6};

std::vector<My_Pair<int, double>> vec;
```

**std::pair**

```
#include <utility>

std::pair<std::string, int> p1 {"Frank", 100};

std::cout << p1.first;      // Frank
std::cout << p1.second;     // 100
```

Figure 22: STL

## Introduction aux conteneurs STL

### Conteneurs

- Structures de données pouvant stocker des objets de « presque » n'importe quel type
  - Classe basée sur un modèle
- Chaque conteneur a des fonctions membres
  - Certains sont spécifiques au conteneur
  - D'autres sont disponibles pour tous les conteneurs
- Chaque conteneur a un fichier d'en-tête associé
  - `#include <container_type>`

### Conteneurs - Commun

| Function | Description |
|---|---|
| `Default constructor` | Initializes an empty container |
| `Overloaded constructors` | Initializes containers with many options |
| `Copy constructor` | Initializes a container as a copy of another container |
| `Move constructor` | Moves existing container to new container |
| `Destructor` | Destroys a container |
| `Copy assignment (operator=)` | Copy one container to another |
| `Move assignment (operator=)` | Move one container to another |
| `size` | Returns the number of elements in the container |
| `empty` | Returns boolean – is the container empty? |
| `insert` | Insert an element into the container |

| Function | Description |
|---|---|
| `operator< and operator<=` | Returns boolean - compare contents of 2 containers |
| `operator> and operator>=` | Returns boolean - compare contents of 2 containers |
| `operator== and operator!=` | Returns boolean - are the contents of 2 containers equal or not |
| `swap` | Swap the elements of 2 containers |
| `erase` | Remove element(s) from a container |
| `clear` | Remove all elements from a container |
| `begin and end` | Returns iterators to first element or end |
| `rbegin and rend` | Returns reverse iterators to first element or end |
| `cbegin and cend` | Returns constant iterators to first element or end |
| `crbegin and crend` | Returns constant reverse iterators to first element or end |

### Conteneur - éléments

- Quels types d'éléments peut-on stocker dans des conteneurs ?
- Une `copie` de l'élément sera stockée dans le conteneur
  - Toutes les primitives OK
- L'élément doit être
  - Copiable et assignable (constructeur de copie / affectation de copie)
  - Mobile pour plus d'efficacité (déplacer le constructeur / déplacer l'affectation)
- Les conteneurs associatifs ordonnés doivent pouvoir comparer des éléments
  - `oparator <, operator ===`

## Introduction aux itérateurs STL

### Itérateurs

- Permet d'abstraire un conteneur arbitraire sous forme de séquence d'éléments
- Ce sont des objets qui fonctionnent comme des pointeurs par conception

- La plupart des classes de conteneurs peuvent être parcourues avec des itérateurs.

**Déclaration des itérateurs**

- Les itérateurs doivent être déclarés en fonction du type de conteneur sur lequel ils vont itérer

```
container_type::iterator_type iterator_name;
```

```
std::vector<int>::iterator it1;
std::list<std::string>::iterator it2;
std::map<std::string, std::string>::iterator it3;
std::set<char>::iterator it4;
```

Figure 23: STL

**Méthodes de début et de fin de l'itérateur**

```
std::vector<int> vec {1,2,3};
```

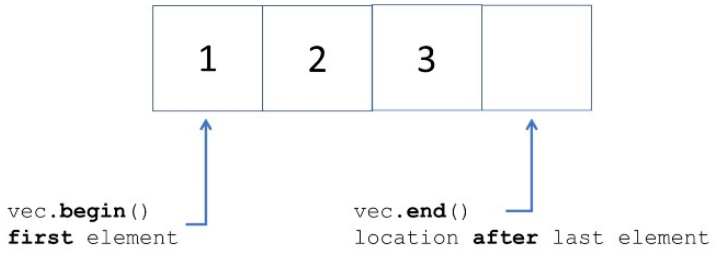



Figure 24: STL

**Déclaration des itérateurs**

**Initialisation des itérateurs**

**Opérations dont les itérateurs (`it`)**

**Utilisation d'itérateurs - `std::vector`**

```
std::set<char> suits {'C','H','S','D'};
```

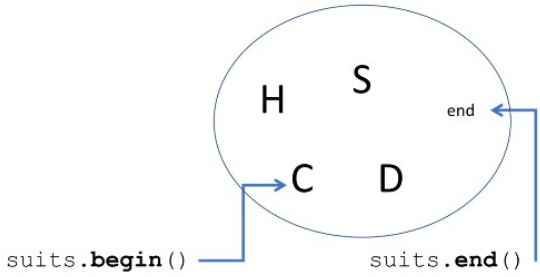



Figure 25: STL

```
std::vector<int> vec {1,2,3};

std::vector<int>::iterator it = vec.begin();

or

auto it = vec.begin();
```

Figure 26: STL

| Operation | Description | Type of Iterator |
|---|---|---|
| `++it` | Pre-increment | All |
| `it++` | Post-increment | All |
| `it = it1` | Assignment | All |
| `*it` | Dereference | Input and Output |
| `it->` | Arrow operator | Input and Output |
| `it == it1` | Comparison for equality | Input |
| `it != it1` | Comparison for inequality | Input |
| `--it` | Pre-decrement | Bidirectional |
| `it--` | Post-decrement | Bidirectional |
| `it + i, it += i`<br>`it - i, it -= i` | Increment and decrement | Random Access |
| `it < it1, it <= it1`<br>`it > it1, it >= it1` | Comparison | Random Access |

Figure 27: STL

```
std::vector<int> vec {1,2,3};

std::vector<int>::iterator it = vec.begin();

while (it != vec.end()) {
   std::cout << *it << " ";
   ++it;
}
// 1 2 3
```

```
std::vector<int> vec {1,2,3};

for (auto it = vec.begin(); it != vec.end(); it++) {
   std::cout << *it << " ";
}
// 1 2 3
```

- Voici comment fonctionne la boucle for basée sur une plage

**Utilisation d'itérateurs - `std::set`**

```
std::set<char> suits {'C', 'H', 'S', 'D'};

auto it = suits.begin();

while (it != suits.end()) {
   std::cout << *it << " " << std::end;
   ++it;
}
// C H S D
```

Figure 28: STL

**Itérateurs inversés**

- Fonctionne à l'envers
- Le dernier élément est le premier et le premier est le dernier
- `++` déplace le backend, `--` avance

**Autres itérateurs**

```
std::vector<int> vec {1,2,3};
std::vector<int>::reverse_iterator it = vec.begin();
while (it != vec.end()) {
   std::cout << *it << " ";
   ++it;
}
// 3 2 1
```

Figure 29: STL

- `begin()` and `end()`
  - `iterator`
- `cbegin()` and `cend()`
  - `const_iterator`
- `rbegin()` and `rend()`
  - `reverse_iterator`
- `crbegin()` and `crend()`
  - `const_reverse_iterator`

Figure 30: STL

## Introduction aux algorithmes `STL`

### Algorithmes

- Les algorithmes `STL` fonctionnent sur des séquences d'éléments de conteneur qui leur sont fournies par un itérateur
- `STL` possède de nombreux algorithmes communs et utiles
- Trop de choses à décrire dans cette section
  - http://en.cppreference.com/w/cpp:algorithm
- De nombreux algorithmes nécessitent des informations supplémentaires pour faire leur travail
  - Foncteurs (objets fonction)
  - Pointeurs de fonction
  - Expressions lambda `(C++11)`

### Algorithmes et itérateurs

```
#include <algorithm>
```

- Différents conteneurs prennent en charge différents types d'itérateurs
  - Détermine les types d'algorithmes pris en charge
- Tous les algorithmes `STL` attendent des itérateurs comme arguments
  - Détermine la séquence obtenue à partir du conteneur

### Invalidation de l'itérateur

- Les itérateurs pointent vers des éléments de conteneur
- Il est possible que les itérateurs deviennent invalides pendant le traitement
- Supposons que nous parcourions un vecteur de 10 éléments
  - Et on `clear()` le vecteur en itérant ? ce qui se produit ?
  - Comportement non défini : nos itérateurs pointent vers des emplacements non valides

### Exemple d'algorithme - `find` avec des types primitifs

- L'algorithme `find` tente de localiser la première occurrence d'un élément dans un conteneur
- Beaucoup de variantes
- Renvoie un itérateur pointant vers l'élément localisé ou `end()`

### Exemple d'algorithme - `find` avec des types définis par l'utilisateur

- Trouver doit pouvoir comparer des objets
- `operators==` est utilisé et doit être fourni par votre classe

### Exemple d'algorithme - `for_each`

- L'algorithme `for_each` applique une fonction à chaque élément de la séquence d'itérateurs.

```
std::vector<int> vec {1,2,3};

auto loc = std::find(vec.begin(), vec.end(), 3);

if (loc != vec.end())              // found it!
   std::cout << *loc << std::endl;  // 3
```

Figure 31: STL

```
std::vector<Player> team { /* assume initialized */ }
Player p {"Hero", 100, 12};

auto loc = std::find(team.begin(), team.end(), p);

if (loc != vec.end())                  // found it!
   std::cout << *loc << std::endl;     // operator<< called
```

Figure 32: STL

- La fonction doit être fournie à l'algorithme comme :
  - Foncteur (objet fonction)
  - Pointeur de fonction
  - Expression lambda `(C++11)`
- Mettons au carré chaque élément

**`for_each` - en utilisant un foncteur**

```
struct Square_Functor {
     void operator()(int x) {   // overload () operator
          std::cout << x * x << " ";
     }
};
Square_Functor square;     // Function object

std::vector<int> vec {1, 2, 3, 4};

std::for_each(vec.begin(), vec.end(), square);
// 1 4 9 16
```

Figure 33: STL

**`for_each` - en utilisant un pointeur de fonction**

**`for_each` - en utilisant une expression lambda**

```
void square(int x) {              // function
     std::cout << x * x << " ";
}

std::vector<int> vec {1, 2, 3, 4};

std::for_each(vec.begin(), vec.end(), square);
// 1 4 9 16
```

Figure 34: STL

```
std::vector<int> vec {1, 2, 3, 4};

std::for_each(vec.begin(), vec.end(),
     [](int x) { std::cout << x * x << " "; }) // lambda

// 1 4 9 16
```

Figure 35: STL

## Conteneur de séquence - Tableau

**`std::array (C++11)`**

`#include <tableau`

- Taille fixe
  - La taille doit être connue au moment de la compilation
- Accès direct aux éléments
- Donne accès au tableau brut sous-jacent
- Utiliser à la place des tableaux bruts lorsque cela est possible
- Tous les itérateurs disponibles et n'invalident pas

**`std::array` - Initialisation et affectation**

```
std::array<int, 5> arr1 { {1,2,3,4,5} };   C++11 vs. C++14

std::array<std::string, 3> stooges {
     std::string("Larry"},
     "Moe",
     std::string("Curly"}
};

arr1 = {2,4,6,8,10};
```

Figure 36: STL

**`std::array` - méthodes courantes**

```
std::array<int, 5> arr {1,2,3,4,5};

std::cout << arr.size();      // 5

std::cout << arr.at(0);       // 1
std::cout << arr[1];          // 2

std::cout << arr.front();     // 1
std::cout << arr.back();      // 5
```

```
std::array<int, 5> arr {1,2,3,4,5};
std::array<int, 5> arr1 {10,20,30,40,50};

std::cout << arr.empty();      // 0 (false)
std::cout << arr.max_size();   // 5

arr.swap(arr1);                // swaps the 2 arrays
int *data = arr.data();        // get raw array address
```

- https://en.cpppreference.com/w/
- https://en.cpppreference.com/w/cpp/container/array
- https://en.cpppreference.com/w/cpp/container/array/front

## Conteneurs de séquence - Vecteur

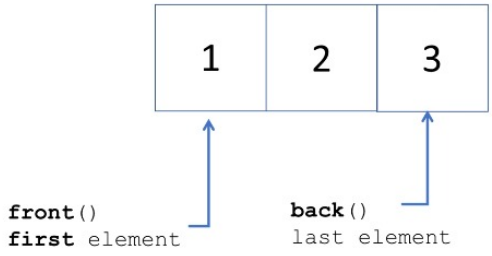
**`std::vecteur`**

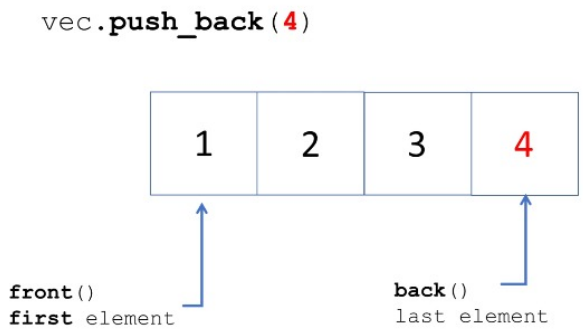
`#include <vecteur>`

- Taille dynamique
  - Hnadled automatiquement
  - Peut s'étendre et se contracter selon les besoins
  - Les éléments sont stockés dans la mémoire contiguë sous forme de tableau
- Accès direct aux éléments (temps constant)
- Insertion et suppression rapides à l'arrière (temps constant)
- Insertion ou suppression d'éléments (temps linéaire)
- Tous les itérateurs disponibles et peuvent invalider

`std :: vecteur`

```
std::vector<int> vec {1,2,3};
```






**std::vector - initialisation et affectation**

```
std::vector<int> vec {1,2,3,4,5};
std::vector<int> vec1 (10,100);  // ten 100s

std::vector<std::string> stooges {
      std::string{"Larry"},
      "Moe",
      std::string{"Curly"}
};

vec1 = {2,4,6,8,10};
```

Figure 37: STL

**std::vector - Méthodes courantes**

```
std::vector<int> vec {1,2,3,4,5};

std::cout << vec.size();          // 5
std::cout << vec.capacity();      // 5
std::cout << vec.max_size;        // a very large number

std::cout << vec.at(0);           // 1
std::cout << vec[1];              // 2

std::cout << vec.front();         // 1
std::cout << vec.back();          // 5
```

```
Person p1 {"Larry", 18};
std::vector<Person> vec;

vec.push_back(p1);       // add p1 to the back
vec.pop_back();          // remove p1 from the back

vec.push_back(Person{"Larry", 18});

vec.emplace_back("Larry", 18);     // efficient!!
```

```
std::vector<int> vec1 {1,2,3,4,5};
std::vector<int> vec2 {10,20,30,40,50};

std::cout << vec1.empty();    // 0 (false)
vec1.swap(vec2);              // swaps the 2 vector

std::sort(vec1.begin(), vec1.end());
```

```
std::vector<int> vec1 {1,2,3,4,5};
std::vector<int> vec2 {10,20,30,40,50};

auto it = std::find(vec1.begin(), vec1.end(), 3);
vec1.insert(it, 10);     // 1,2,10,3,4,5

it = std::find(vec1.begin(), vec1.end(), 4);
vec1::insert(it, vec2.begin(), vec2.end());
     // 1,2,10,3,10,20,30,40,50,4,5
```

## Conteneurs de séquence - Deque

### `std::deque` (file d'attente à double extrémité)

`#include <deque>`

- Taille dynamique
  - Hnadled automatiquement
  - Peut s'étendre et se contracter selon les besoins
  - Les éléments ne sont PAS stockés dans la mémoire contiguë.
- Accès direct aux éléments (temps constant)
- Insertion et suppression rapides au recto et au verso (temps constant)
- Insertion ou suppression d'éléments (temps linéaire)

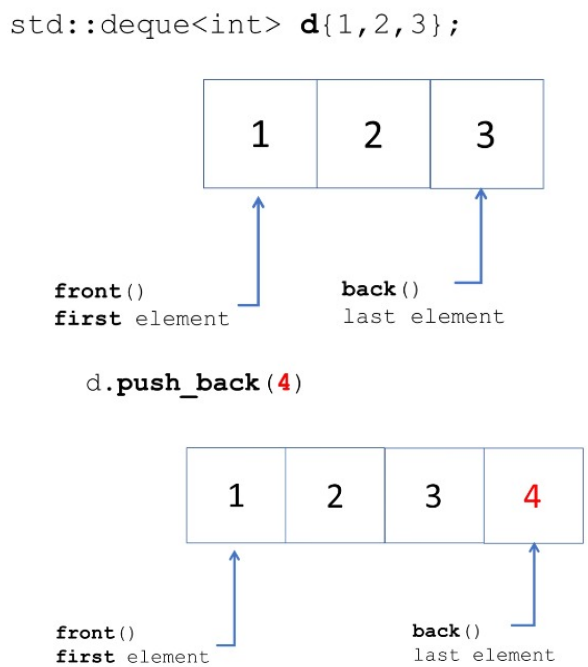
- Tous les itérateurs disponibles et peuvent invalider

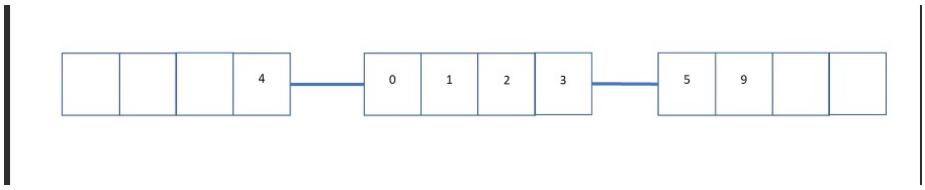
**std::deque - initialisation et affectation**

```
std::deque<int> d{1,2,3,4,5};
std::deque<int> d1(10,100);      // ten 100s

std::deque<std::string> stooges {
      std::string("Larry"},
      "Moe",
      std::string("Curly"}
};

d = {2,4,6,8,10};
```

Figure 38: STL

**std::deque**



**`std::deque` - méthodes courantes**

```
std::deque<int> d {1,2,3,4,5};

std::cout << d.size();          // 5
std::cout << d.max_size();      // a very large number

std::cout << d.at(0);           // 1
std::cout << d[1];              // 2

std::cout << d.front();         // 1
std::cout << d.back();          // 5
```

```
Person p1 {"Larry", 18};
std::deque<Person> d;

d.push_back(p1);           // add p1 to the back
d.pop_back();              // remove p1 from the  back

d.push_front(Person{"Larry", 18});
d.pop_front();             // remove element from the front

d.emplace_back("Larry", 18);     // add to back efficient!!
d.emplace_front("Moe", 24);      // add to front
```

## Sequence Containers - List and Forward List

`std :: list`

`#include <list>`

- Taille dynamique
  - Liste des éléments
  - `list` est bidirectionnelle (doublement liée)
- L'accès aux éléments dynamiques n'est PAS fourni
- Insertion et suppression rapides d'éléments n'importe où dans le conteneur (temps constant)
- Tous les itérateurs disponibles et invalidés lorsque l'élément correspondant est supprimé

`std :: list`

**`std::list` - initialisation et affectation**

**`std::list` - méthodes courantes**

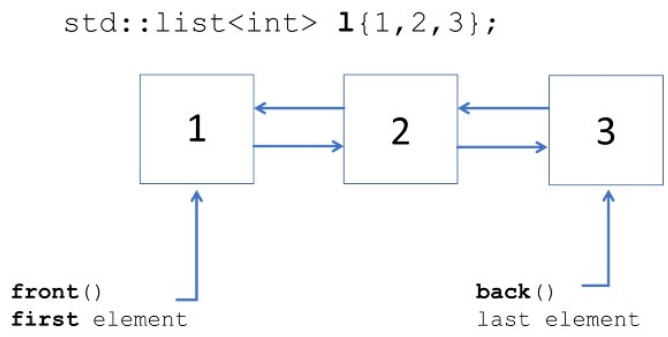



Figure 39: STL

```
std::list<int> l {1,2,3,4,5};
std::list<int> l1(10,100); // ten 100s

std::list<std::string> stooges {
      std::string{"Larry"},
      "Moe",
      std::string{"Curly"}
};

l = {2,4,6,8,10};
```

Figure 40: STL

```
std::list<int> l {1,2,3,4,5};

std::cout << l.size();        // 5
std::cout << l.max_size();    // a very large number

std::cout << 1.front();       // 1
std::cout << 1.back();        // 5


Person p1 {"Larry", 18};
std::list<Person> l;

l.push_back(p1);            // add p1 to the back
l.pop_back();               // remove p1 from the  back

l.push_front(Person{"Larry", 18});
l.pop_front();              // remove element from the front

l.emplace_back("Larry", 18);     // add to back efficient!!
l.emplace_front("Moe", 24);      // add to front
```

### `std::list` - méthodes qui utilisent des itérateurs

```
std::list<int> l {1,2,3,4,5};
auto it = std::find(l.begin(), l.end(), 3);

l.insert(it, 10);          // 1 2 10 3 4 5

l.erase(it);               // erases the 3,  1 2 10 4 5

l.resize(2);               // 1 2

l.resize(5);               // 1 2 0 0 0
```

Figure 41: STL

### `std::list` - méthodes courantes

### `std::forward_list`

`#include <forward_list>`

- Taille dynamique
  - Listes d'éléments
  - `list` unidirectionnelle (uniquement liée)
  - Moins saturé qu'un `std::list`
- L'accès direct aux éléments n'est PAS fourni

```
// traversing the list (bi-directional)

std::list<int> l {1,2,3,4,5};
auto it = std::find(l.begin(), l.end(), 3);

std::cout << *it;             // 3
it++;
std::cout << *it;             // 4
it--;
std::cout << *it;             // 3
```

Figure 42: STL

- Insertion et suppression rapides d'éléments n'importe où dans le conteneur (temps constant)
- Itérateurs inversés non disponibles. Les itérateurs sont invalidés lorsque l'élément correspondant est supprimé

**`std::forward_list`**

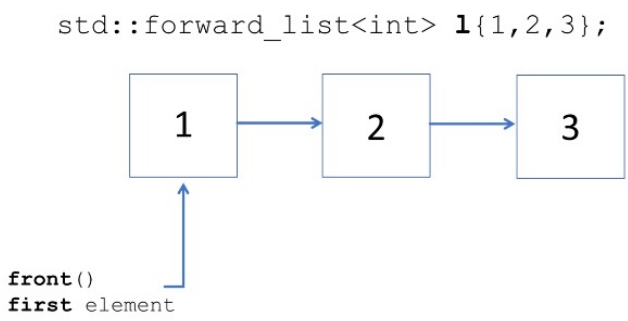



Figure 43: STL

**`std::forward_list` - méthodes courantes**

**`std::forward_list` - méthodes qui utilisent des itérateurs**

```
std::forward_list<int> l {1,2,3,4,5};

std::cout << l.size();          // Not available
std::cout << l.max_size;        // a very large number

std::cout << l.front();         // 1
std::cout << l.back();          // Not available
```

Figure 44: STL

```
Person p1 {"Larry", 18};
std::forward_list<Person> l;

l.push_front(p1);          // add p1 to the front
l.pop_front();             // remove p1 from the front


l.emplace_front("Moe", 24};          // add to front
```

```
std::forward_list<int> l {1,2,3,4,5};
auto it = std::find(l.begin(), l.end(), 3);

l.insert_after(it, 10);     // 1 2 3 10 4 5
l.emplace_after(it, 100);   // 1 2 3 100 10 4 5

l.erase_after(it);          // erases the 100,  1 2 3 10 4 5

l.resize(2);                // 1 2

l.resize(5);                // 1 2 0 0 0
```

## Conteneurs associatifs - Ensembles

- Les conteneurs `STL Set`
- Conteneurs associatifs
  - Collection d'objets stockés qui permettent une récupération rapide à l'aide d'une clé
  - `STL` fournit des `Sets` et `Maps`
  - Généralement implémenté sous la forme d'un « arbre » ou de « hash-sets » binaires équilibrés
  - La plupart des opérations sont très efficaces

- Sets
  - `std::set`
  - `std::unordered_set`
  - `std::multiset`
  - `std::unordered_multiset`

`std :: set`

`#include <ensemble>`

- Semblable à un ensemble mathématique
- Commandé par clé
- Aucun élément en double
- Tous les itérateurs disponibles et invalidés lorsque l'élément correspondant est supprimé

**`std::set` - initialisation et affectation**

```
std::set<int> s {1,2,3,4,5};

std::set<std::string> stooges {
     std::string{"Larry"},
     "Moe",
     std::string{"Curly"}
};

s = {2,4,6,8,10};
```

Figure 45: STL

**`std::set` - méthodes courantes**

```
std::set<int> s {4,1,1,3,3,2,5};   // 1 2 3 4 5

std::cout << s.size();         // 5
std::cout << s.max_size;       // a very large number
```

- No concept of front and back

```
s.insert(7);                   // 1 2 3 4 5 7
```

```
Person p1 {"Larry", 18};
Person p2 {"Moe", 25};

std::set<Person> stooges;

stooges.insert(p1);                // adds p1 to the set
auto result = stooges.insert(p2);   // adds p2 to the set
```

- Utilise `operator<` pour commander !
- renvoie un `std::pair<iterator, bool>`
  - `find` est un itérateur vers l'élément inséré ou vers le doublon dans l'ensemble
  - `second` est un booléen indiquant le succès ou l'échec

```
std::set<int> s {1,2,3,4,5};


s.erase(3);              // erase the 3 : 1 2 4 5

auto it = s.find(5);
if (it != s.end())
   s.erase(it);          // erase the 5: 1 2 4
```

```
std::set<int> s {1,2,3,4,5};

int num = s.count(1);     // 0 or 1

s.clear();                // remove all elements

s.empty();                // true or false
```

`std ::unordered_set`

`#include <unordered_set>`

- Les éléments ne sont pas ordonnés
- Aucun élément en double autorisé
- Les éléments ne peuvent pas être modifiés
  - Doit être effacé et un nouvel élément installé
- Aucun itérateur inverse n'est autorisé

`std ::unordered_multiset`

`#include <unordered_set>`

- Les éléments sont illimités
- Permet les éléments en double
- Aucun itérateur inverse n'est autorisé

## Conteneurs associatifs - Maps

`std::map`

`#include <map>`

- Semblable à un dictionnaire
- Les éléments sont stockés sous forme de paires clé-valeur `(std::pair)`
- Commandé par clé
- Aucun élément en double (les clés sont uniques)
- Accès direct aux éléments à l'aide de la clé
- Tous les itérateurs et invalidés lorsque les éléments correspondants sont supprimés

### `std::map` - initialisation et affectation

```
std::map<std::string, int> m1 {
   {"Larry", 18},
   {"Moe", 25}
};

std::map<std::string, std::string> m2 {
   {"Bob", "Butcher"},
   {"Anne", "Baker"},
   {"George", "Candlestick maker"}
};
```

Figure 46: STL

### `std::map` - méthodes courantes

```
std::map<std::string, std::string> m {
   {"Bob", "Butcher"},
   {"Anne", "Baker"},
   {"George", "Candlestick maker"}
};

std::cout << m.size();          // 3
std::cout << m.max_size;        // a very large number
```

```
std::map<std::string, std::string> m {
   {"Bob", "Butcher"},
   {"Anne", "Baker"},
   {"George", "Candlestick maker"}
};

std::pair<std::string, std::string> p1 {"James", "Mechanic"};

m.insert(p1);

m.insert(std::make_pair("Roger", "Ranger"));
```

```
std::map<std::string, std::string> m {
   {"Bob", "Butcher"},
   {"Anne", "Baker"},
   {"George", "Candlestick maker"}
};

m["Frank"] = "Teacher";        // insert

m["Frank"] = "Instructor";     // update value
m.at("Frank") = "Professor";   // update value
```

```
std::map<std::string, std::string> m {
   {"Bob", "Butcher"},
   {"Anne", "Baker"},
   {"George", "Candlestick maker"}
};

m.erase("Anne");                        // erase Anne

if (m.find("Bob") != m.end())           // find Bob
      std::cout << "Found Bob!";

auto it = m.find("George");
if (it != m.end())
   m.erase(it);                         // erase George
```

```
std::map<std::string, std::string> m {
   {"Bob", "Butcher"},
   {"Anne", "Baker"},
   {"George", "Candlestick maker"}
};

int num = m.count("Bob");  // 0 or 1

m.clear();                 // remove all elements

m.empty();                 // true or false
```

std ::unordered_map

`#include <carte_non ordonnée>`

- Les éléments ne sont pas ordonnés
- Aucun élément en double autorisé
- Aucun itérateur inverse n'est autorisé

## Adaptateurs de conteneur - Pile

`std :: pile`

- Structure de données Last-In-First-Out (LIFO)
- Implémenté en tant qu'adaptateur sur un autre conteneur `STL`. Peut être implémenté en tant que `vecteur, liste` ou `deque`.
- Toutes les opérations se produisent à une extrémité de la « pile » (en haut).
- Aucun itérateur n'est pris en charge

`std :: pile`

`#include <pile>`

- `push` - insère un élément en haut de la pile
- `pop` - supprime un élément du haut de la pile
- `top` - accède à l'élément supérieur de la pile
- `empty` - la pile est-elle vide ?
- `size` - nombre d'éléments dans la pile

**`std::stack` - Initialisation**

```
std::stack<int> s;                        // deque

std::stack<int, std::vector<int>> s1;     // vector

std::stack<int, std::list<int>> s2;       // list

std::stack<int, std::deque<int>> s3;      // deque
```

Figure 47: STL

**`std::stack` - Méthodes courantes**

```
std::stack<int> s;

s.push(10);
```

```
s.push(20);
```

```
s.push(30);
```

```
std::cout << s.top();
```

```
// 30

s.pop();
```

```
// 30 is removed

s.pop();
```

```
// 20 is removed
std::cout << s.size();
// 1
```

## Adaptateurs de conteneur – File d'attente

`std::queue`

- Structure de données premier entré, premier sorti (FIFO)
- Implémenté en tant qu'adaptateur sur un autre conteneur `STL`. Peut être implémenté en tant que `list` ou `deque`.
- Les éléments sont poussés au niveau du backend et extraits de la police.
- Aucun itérateur n'est pris en charge

`std::queue`

`#include <queue>`

- `push` - insère un élément à la fin de la file d'attente
- `pop` - supprime un élément de la police de la file d'attente
- `front` - accéder à l'élément à l'avant
- `back` - accéder à l'élément à l'arrière
- `empty` - la file d'attente est-elle vide ?
- `size` - nombre d'éléments dans la file d'attente

### `std::queue` - Initialisation

### `std::queue` - Méthodes courantes

```
std::queue<int> q;                          // deque

std::queue<int, std::list<int>> q2;         // list

std::queue<int, std::deque<int>> q3;        // deque
```

Figure 48: STL

```
std::queue<int> q;

q.push(10);                                 10

q.push(20);                                 20 10

q.push(30);                                 30 20 10

std::cout << q.front();                     30 20 10   // 10

std::cout << q.back();                      30 20 10   // 30

q.pop();                                    30 20      // remove 10

q.pop();                                    30         // remove 20

std::cout << q.size();                                 // 1
```

## Adaptateurs de conteneurs - File d'attente prioritaire

`std ::priority_queue`

- Permet l'insertion et le retrait d'éléments dans l'ordre depuis l'avant du conteneur
- Les éléments sont stockés en interne sous forme de vecteur par défaut
- Les éléments sont insérés dans l'ordre « prioritaire » (la plus grande valeur sera toujours au premier plan)
- Aucun itérateur n'est pris en charge

`std::priority_queue`

`#include <queue>`

- `push` - insère un élément dans l'ordre trié
- `pop` - supprime l'élément supérieur (le plus grand)
- `top` - accéder à l'élément supérieur (le plus grand)
- `empty` - la file d'attente est-elle vide ?
- `size` - nombre d'éléments dans la file d'attente

**`std::priority_queue` - Initialisation**

```
std::priority_queue<int> pq;          // vector

pq.push(10);
pq.push(20);
pq.push(3);
pq.push(4);

std::cout << pq.top();                // 20 (largest)
pq.pop();                             // remove 20
pq.top();                             /  10 (largest)
```

Figure 49: STL